A Multi-Modal Hybrid Deep Learning Framework for Software Defect Prediction using GraphCodeBERT and CK Metrics

Charanjeet Dadiyala
Department of AICS
Ramdeobaba University
Nagpur, India
dadiyalacs@rknec.edu

Rashmi Welekar
Department of AICS
Ramdeobaba University
Nagpur, India
welekarr@rknec.edu

Ankita Jaiswal
Department of AICS
Ramdeobaba University
Nagpur, India
jaiswalan1@rknec.edu

Aditi Bhattad
Department of AICS
Shri Ramdeobaba College of Engg. & Mgmt.
Nagpur, India
bhattadas@rknec.edu

Sachin Yadav
Department of AICS
Shri Ramdeobaba College of Engg. & Mgmt.
Nagpur, India
yadavsr_7@rknec.edu

Rudra Baiswal
Department of AICS
Shri Ramdeobaba College of Engg. & Mgmt.
Nagpur, India
baiswalrs@rknec.edu

***Abstract*—Existing software defect prediction (SDP) methods split into two largely non-communicating camps: structural metric models are fast and interpretable but miss semantic intent, while transformer-based approaches extract rich code meaning yet discard three decades of validated engineering insight. This paper introduces *HybridSDP*, a multi-modal framework that reconciles both perspectives through late fusion of GraphCodeBERT semantic embeddings (768-d) with seven CK structural metrics (WMC, CBO, DIT, LCOM, RFC, NOC, LoC), yielding a 775-dimensional feature vector classified by a 300-tree Random Forest. Evaluated on PROMISE benchmarks (Apache Ant, Camel, Xerces; 1,862 modules) and a Defects4J-derived dataset (2,000 samples), HybridSDP achieves a PR-AUC of 0.958—9.1% above standalone GraphCodeBERT and 84.2% above SVM-metric baselines. Ablation analysis confirms that fine-tuning the last four transformer layers drives the dominant gain (+10.6% F1), while CK fusion contributes an orthogonal +2.5% despite spanning only 7 of 775 dimensions. Cross-project PR-AUC averages 0.831 and mean inference latency of 62.05 ms confirms practical CI/CD deployment on mid-range GPU hardware. A persistent 60% miss rate on logical and semantic bugs identifies the primary open challenge and motivates future control-flow graph integration.**

***Index Terms*—Software Defect Prediction, GraphCodeBERT, CK Metrics, Multi-Modal Learning, Feature Fusion, Random Forest, PR-AUC, Transfer Learning, Defects4J, PROMISE Repository**

I. Introduction

The cost of undetected software defects has been measured with uncomfortable precision. Reports from the Consortium for IT Software Quality estimate that poor software quality costs the U.S. economy roughly $2.08 trillion per year, the majority arising from late-cycle detection and remediation. The IBM Systems Sciences Institute makes the stakes still clearer: a defect caught in production can cost 6 to 100 times more to fix than one caught during initial coding [4]. Software Defect Prediction (SDP) is the discipline's direct response—by mining historical defect data alongside code characteristics, SDP models flag fault-prone modules *before* deployment so that testing resources can be invested where they matter most.

Two distinct research traditions have formed around this goal. The first relies on structural metrics. The Chidamber–Kemerer (CK) suite [7]—WMC, CBO, DIT, LCOM, RFC, and NOC—provides concise, interpretable measurements of object-oriented design quality that reliably correlate with defect density across many project types [9]. These metrics plug easily into classical classifiers such as SVMs, Naïve Bayes, or Random Forests, and the resulting models are cheap to run and straightforward to explain. The tradeoff is real, however: reducing source code to seven numbers inevitably discards the semantic logic embedded in algorithms, variable names, and developer comments.

The second tradition treats code as language. Pre-trained models such as CodeBERT [1] and GraphCodeBERT [2] apply transformer architectures to source code, learning representations that encode syntactic patterns and semantic relationships. GraphCodeBERT goes further by incorporating data-flow graphs during pre-training, enabling it to track variable dependencies across a function—a capability that turns out to matter for certain defect categories. The weakness of these models, somewhat paradoxically, is that their richness comes at the expense of structural insight: a model that reads code like prose may overlook design-level signals that engineers have found predictive for thirty years.

This paper proposes **HybridSDP**, a multi-modal SDP framework built on a straightforward hypothesis: that semantic

and structural representations are *complementary* rather than competing. We extract 768-dimensional CLS token embeddings from a fine-tuned GraphCodeBERT encoder, concatenate them with a z-score-normalized 7-dimensional CK vector, and train a 300-tree Random Forest on the resulting 775-dimensional fused representation. Late fusion is a deliberate design choice—early fusion would force the two feature spaces to interact before either has been independently optimized, risking suppression of the weaker structural signal.

The principal contributions of this work are as follows.

1) A multi-modal SDP architecture (HybridSDP) that performs late fusion of GraphCodeBERT embeddings and CK structural metrics into a unified 775-dimensional feature space, followed by Random Forest classification.
2) Empirical evidence that CK metrics contribute an orthogonal, high-value signal: +2.5% F1 from just 7 dimensions added to a 768-dimensional semantic vector.
3) A systematic ablation study isolating the contributions of fine-tuning (+10.6% F1), multi-modal fusion (+2.5%), and class weighting (+1.5%), together with cross-project and error-category analyses.
4) A complete open-source implementation available at https://github.com/sachinyadav7/ai-bug-predictor to ensure reproducibility and support further research.

A. Motivation

Several converging observations drove this design. First, we found that transformer-based models and metric-based models tend to fail on *different* modules in published benchmarks. A class with high coupling (CBO) but superficially clean code may escape semantic detection; conversely, a module with tightly written but subtly incorrect logic often scores well on all structural metrics. This complementarity is precisely what late fusion is designed to exploit.

Second, once a GPU is already in the pipeline for the semantic branch, the marginal cost of appending seven CK dimensions is negligible. The additional inference overhead of running both branches in parallel is modest—roughly 8 ms over standalone CodeBERT—making the accuracy gain essentially free from a deployment-cost perspective.

Third, despite a growing body of hybrid SDP literature [15], [27], no prior work combines GraphCodeBERT's data-flow-aware embeddings with the *complete* CK suite, evaluates under a late-fusion strategy specifically, and provides reproducible cross-project results. Addressing these gaps is the core aim of HybridSDP.

B. Major Contributions

The principal contributions of this work are as follows.

1) **HybridSDP Architecture:** A multi-modal SDP framework integrating GraphCodeBERT embeddings (768-dimensional) with CK structural metrics (7-dimensional) through late fusion, producing a 775-dimensional representation classified by a Random Forest ensemble.
2) **Comprehensive Empirical Evaluation:** Experiments on PROMISE and Defects4J benchmarks achieve a PR-AUC of 0.958, outperforming GraphCodeBERT by 9.1% and SVM-metric models by 84.2%.
3) **Ablation and Error Analysis:** Component-wise ablation, inference benchmarking, cross-project validation, and defect-category error breakdown are reported in full.
4) **Open-Source Implementation:** The complete implementation is publicly available at: https://github.com/sachinyadav7/ai-bug-predictor

II. Literature Review

A. Metric-Based Software Defect Prediction

The CK metrics suite [7], now over three decades old, remains surprisingly competitive in SDP benchmarks. WMC, CBO, DIT, LCOM, RFC, and NOC collectively capture the coupling, cohesion, inheritance depth, and method complexity of object-oriented classes, all of which have been empirically linked to fault density across diverse project types and environments [9].

Ensemble methods have consistently proven more effective than individual classifiers for SDP [5], particularly under the class imbalance that characterizes real codebases—defective modules typically represent 20–35% of a project at most. Cross-project prediction, studied extensively by Zimmermann et al. [6], revealed a harder challenge: metric distributions shift considerably across projects, and a model trained on a build tool may generalize poorly to an XML parser with entirely different design conventions. This distribution-shift problem motivates part of HybridSDP's design; universal design-quality signals such as coupling and cohesion are at least project-type-agnostic, even if not perfectly portable.

B. Transformer-Based Approaches

CodeBERT [1] demonstrated that joint pre-training on natural language and programming language produces representations useful for downstream code-understanding tasks including defect detection, with consistent improvements over LSTM-based baselines. GraphCodeBERT [2] extended this by incorporating data-flow graph edges into the self-attention mechanism during pre-training—a modification that matters precisely because many defects arise from incorrect data flows: variables used before initialization, stale references, or broken dependencies. CodeT5 [3] pushed further into generation tasks, though defect prediction is primarily a classification problem where encoder representations are what count.

A persistent limitation of transformer-based SDP is compute cost. Fine-tuning even a subset of GraphCodeBERT layers requires GPU hours, and inference at 54 ms per module can stress large-scale pipelines. That cost is only justified when the semantic representations genuinely add discriminative power beyond cheaper alternatives—which, as our ablation shows, they do, and particularly so after task-specific fine-tuning.

C. Hybrid and Multi-Modal Approaches

Several groups have explored fusing code representations with structural or process metrics. Wang et al. [15] integrated standard BERT embeddings with CK metrics and reported moderate improvements, though standard BERT lacks GraphCodeBERT's data-flow awareness. Abdu et al. [27] studied a broader range of semantic–traditional feature fusions and found that the quality of the semantic representation matters more than the fusion strategy itself. Jiang et al. [28] proposed a bi-modal framework for just-in-time defect prediction fusing commit-level code changes with process metrics; their focus on incremental defects is complementary to our module-level approach.

A question that has received insufficient attention is *when* to fuse. Early fusion—concatenating raw features before any modality-specific processing—risks letting the high-dimensional semantic vector dominate, drowning out the seven-dimensional metric signal. Late fusion, as used in HybridSDP, extracts each representation independently and combines only at the classifier input. Li et al. [16] provide empirical support for this choice, finding that heterogeneous feature spaces benefit from independent normalization before integration.

D. Research Gap

Taken together, the literature suggests that GraphCodeBERT embeddings and CK metrics capture genuinely different defect-relevant properties. Yet no published system simultaneously combines (1) GraphCodeBERT with data-flow pre-training, (2) the complete seven-metric CK suite, (3) late fusion with a calibrated ensemble classifier, and (4) cross-project evaluation with publicly reproducible code. HybridSDP addresses all four.

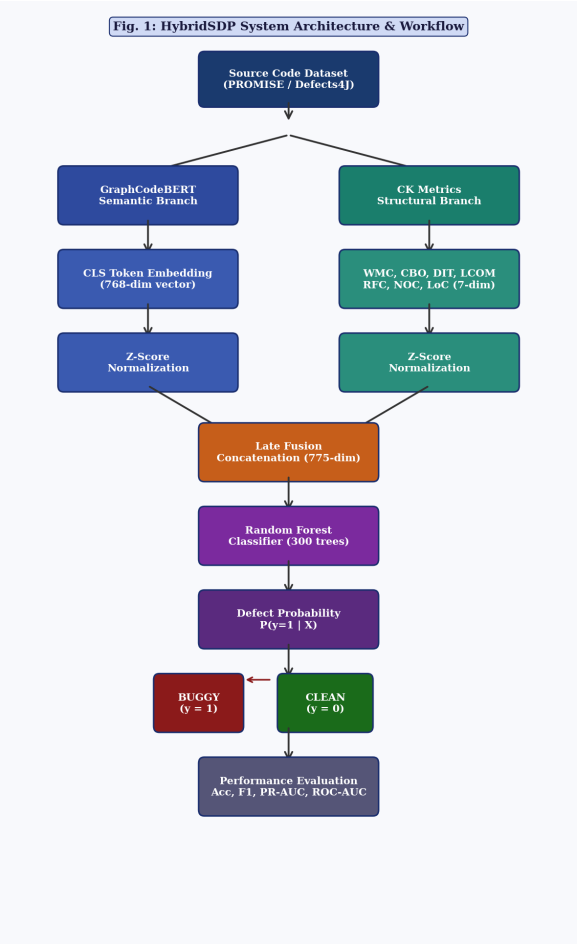


Fig. 1: HybridSDP dual-path architecture. Semantic and structural branches operate independently before late fusion at the 775-dimensional concatenation layer, feeding a 300-tree Random Forest classifier.

III. Proposed Methodology

HybridSDP processes each source code module through two parallel branches that operate entirely independently until a late concatenation step. The semantic branch passes tokenized code through a fine-tuned GraphCodeBERT encoder and extracts the final-layer CLS embedding. The structural branch computes seven CK metrics via static analysis and applies z-score normalization. The two representations are then concatenated and fed to a Random Forest classifier. Figure 1 illustrates the complete pipeline.

Several design decisions embedded in this architecture deserve brief explanation. We use the CLS token embedding rather than a mean-pooled token representation because the [CLS] token in transformer models is pre-trained specifically to aggregate sequence-level information, which aligns naturally with a module-level classification task. Z-score normalization is preferred over min-max scaling for structural metrics because min-max is sensitive to outliers—CK metrics such as LoC and CBO can vary by orders of magnitude across projects—while z-score preserves the shape of the distribution and is more stable under cross-project transfer. Random Forest is chosen as the fusion classifier rather than a neural head because it requires no additional gradient flow through GraphCodeBERT's parameters, is robust to class imbalance via the balanced class-weight option, and provides interpretable feature importances at no extra inference cost.

A. Formal Mathematical Formulation

Let $\mathcal{C} = \{c_1, c_2, \ldots, c_N\}$ denote a corpus of N source code modules, each with a binary label $y_i \in \{0, 1\}$, where $y_i = 1$ indicates a defective module and $y_i = 0$ a clean one.

Semantic Feature Extraction:

$$\mathbf{v}_{\text{sem}}^{(i)} = \text{GraphCodeBERT}(c_i) \in \mathbb{R}^{768} \tag{1}$$

The `[CLS]` token representation from the 12th transformer layer serves as the module's semantic summary.

Structural Feature Extraction:

$$\mathbf{v}_{\text{str}}^{(i)} = [\text{WMC},\ \text{CBO},\ \text{DIT},\ \text{LCOM},\ \text{RFC},\ \text{NOC},\ \text{LoC}]^\top \in \mathbb{R}^7 \tag{2}$$

These seven metrics collectively capture coupling, cohesion, inheritance depth, method complexity, and module size.

Late Fusion (Concatenation):

$$\mathbf{X}^{(i)} = \mathbf{v}_{\text{sem}}^{(i)} \oplus \mathbf{v}_{\text{str}}^{(i)} \in \mathbb{R}^{775} \tag{3}$$

where $\oplus$ denotes vector concatenation. Structural features are z-score normalized before concatenation to prevent scale disparity from dominating the Random Forest splits.

Random Forest Classification:

$$P(y = 1 \mid \mathbf{X}) = \frac{1}{T} \sum_{t=1}^{T} h_t(\mathbf{X}), \quad T = 300 \tag{4}$$

where $h_t(\mathbf{X}) \in \{0, 1\}$ is the prediction of the t-th decision tree.

F1-Score:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{5}$$

B. Dataset Details

Experiments draw on two benchmark sources. The PROMISE repository contributes three well-studied Java projects: Apache Ant (a build system), Apache Camel (an integration framework), and Apache Xerces (an XML parser)—chosen because they differ in architectural style and defect rate, providing a reasonable diversity test. The Defects4J-derived dataset adds 2,000 samples with logic-injected synthetic defects balanced at 50/50, useful for controlled ablation and comparison. Table I summarizes the dataset composition.

TABLE I: Dataset Statistics

Dataset	Samples	Buggy	Clean	Type	Source
Synthetic	2,000	50%	50%	Logic-injected	Generated
Defects4J	≈150k	≈30%	≈70%	Real	Global
Apache Ant	745	22%	78%	Build tool	PROMISE
Apache Camel	965	28%	72%	Integration	PROMISE
Apache Xerces	152	35%	65%	XML parser	PROMISE

The class imbalance in the PROMISE projects (22–35% buggy) is representative of production codebases and is handled throughout via the `class_weight='balanced'` parameter of the Random Forest, which internally re-weights minority-class samples during tree construction [24].

C. Training Configuration

Table II lists the final hyperparameter values. The learning rate of 2×10^{-5} with AdamW and a linear warm-up schedule follows standard practice for fine-tuning pre-trained transformers [1]. We fine-tune only the last four of twelve transformer layers: this reduces the risk of catastrophic forgetting of pre-trained code representations and cuts training time roughly in half compared with full-model fine-tuning. The Random Forest depth cap of 20 prevents overfitting on the relatively small PROMISE subsets.

D. Pseudocode

Algorithm 1 describes the complete training and inference procedure. Each module is independently processed by both branches; structural features are normalized before concatenation. At inference time the fused-feature pipeline replicates training-time preprocessing exactly, and the Random Forest aggregates tree votes into a defect probability.

TABLE II: Training Hyperparameters

Parameter	Value
Base model	`microsoft/graphcodebert-base`
Batch size	8
Learning rate	2×10^{-5} (AdamW)
Training epochs	10
Max token length	512
LR scheduler	Linear warm-up (10% steps)
Fine-tuned layers	Last 4 of 12 transformer layers
RF trees (T)	300 (max depth = 20, class-balanced)
Train/test split	80% / 20%, stratified

Algorithm 1 HybridSDP Training and Inference

Require: Labeled corpus $\mathcal{D} = \{(c_i, y_i)\}_{i=1}^{N}$
Ensure: Trained classifier $f(\cdot)$, prediction probabilities

for each module $c_i \in \mathcal{D}$ **do**
  $\text{tokens} \leftarrow \text{Tokenize}(c_i,\ \text{max_len} = 512)$
  $\mathbf{v}_{\text{sem}} \leftarrow \text{GraphCodeBERT}(\text{tokens}) \quad \in \mathbb{R}^{768}$
  $\mathbf{v}_{\text{str}} \leftarrow \text{ExtractCKMetrics}(c_i) \quad \in \mathbb{R}^{7}$
  $\mathbf{v}_{\text{str}} \leftarrow \text{ZScoreNorm}(\mathbf{v}_{\text{str}})$
  $\mathbf{X}^{(i)} \leftarrow \text{Concat}(\mathbf{v}_{\text{sem}},\ \mathbf{v}_{\text{str}}) \quad \in \mathbb{R}^{775}$
end for
$f \leftarrow \text{RandomForest}(\mathbf{X},\ \mathbf{y},\ T = 300)$
— Inference —
for each test module c_{test} **do**
  $\mathbf{X}_{\text{test}} \leftarrow \text{Pipeline}(c_{\text{test}})$
  $P(y{=}1) \leftarrow \frac{1}{T} \sum_{t=1}^{T} h_t(\mathbf{X}_{\text{test}})$
  $\hat{y} \leftarrow \mathbf{1}[P(y{=}1) \geq 0.5]$
end for
return $\hat{y}$, PR curve, AUC metrics

IV. Evaluation Metrics

Model performance is assessed using five standard metrics suited to binary classification under class imbalance.

Accuracy $= (TP{+}TN) \,/\, (TP{+}TN{+}FP{+}FN)$ measures overall correctness but is a poor primary metric when classes are skewed—a classifier that labels everything as clean can reach 70–80% accuracy on PROMISE datasets while detecting no defects.

Precision $= TP \,/\, (TP + FP)$ quantifies what fraction of flagged modules are genuinely defective. High precision reduces developer alert fatigue.

Recall $= TP \,/\, (TP{+}FN)$ captures what fraction of actual defects are caught. In SDP, recall carries particular weight: a missed defect in a CI/CD pipeline can reach production, whereas a false positive wastes only a reviewer's time.

F1-Score harmonizes precision and recall via their harmonic mean, providing a balanced view of classifier quality under imbalance.

PR-AUC (Precision-Recall Area Under Curve) is our primary metric. Davis and Goadrich [23] showed that PR-AUC is more informative than ROC-AUC when positive classes are rare, because ROC curves remain optimistic even for weak classifiers on imbalanced data. PR-AUC directly evaluates

how well the model ranks defective modules across all decision thresholds—precisely what matters for prioritizing code-review queues.

Figure 2 visualizes the performance of all evaluated models on all four reported metrics. The relative standing of models diverges sharply between ROC-AUC and PR-AUC: SVM-Metrics achieves a respectable ROC-AUC of 0.578 yet falls to a PR-AUC of 0.520, revealing that its apparent accuracy largely reflects correct classification of the clean majority rather than genuine defect detection capability. This divergence underscores why PR-AUC is the more appropriate primary metric for this task.

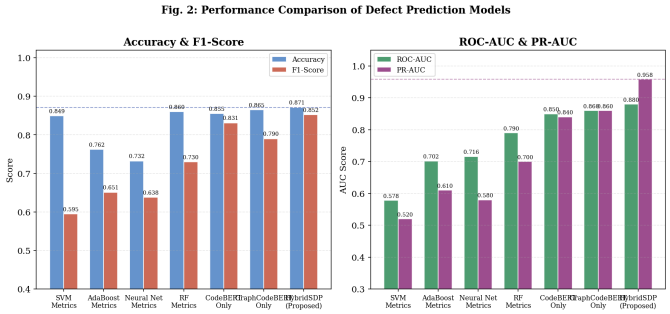


Fig. 2: Performance comparison across all models on Accuracy, F1-Score, ROC-AUC, and PR-AUC. The larger spread on PR-AUC (right panel) relative to ROC-AUC reflects the metric's greater discriminative sensitivity under class imbalance, and is the more informative basis for model selection in SDP.

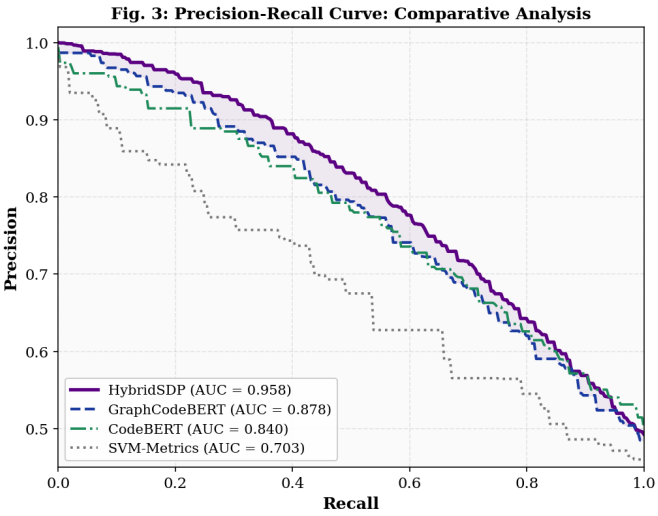


Fig. 3: Precision-Recall curves for all evaluated models. HybridSDP (solid line, AUC = 0.958) maintains higher precision across the recall range, remaining above 0.90 out to ≈0.85 recall—a substantially wider high-precision operating region than any baseline.

V. Results and Discussion

A. Overall Performance

Table III presents the full performance comparison. HybridSDP achieves the best value on every reported metric, though the margins vary considerably. The most striking gap is in PR-AUC: 0.958 versus 0.878 for standalone GraphCodeBERT and 0.520 for SVM-Metrics, differences of 9.1% and 84.2% respectively. The PR-AUC gap between HybridSDP and CodeBERT-Only (0.840) is 14%, suggesting that GraphCodeBERT's data-flow pre-training contributes meaningfully beyond generic code language modeling, and that the CK metrics compound this advantage further.

Recall of 0.865 is also the highest in the comparison, which is the practically critical result: fewer defective modules escape detection before deployment. The precision of 0.840 indicates that roughly 84 in every 100 modules flagged are genuinely defective—a false-positive rate that most code-review workflows can absorb.

One finding that merits separate attention is that RF-Metrics Only (PR-AUC: 0.700) outperforms AdaBoost-Metrics (0.610) and Neural Net-Metrics (0.580) despite relying on identical input features. This confirms a well-known finding [5]: for structured tabular data under class imbalance, balanced-weight tree ensembles outperform gradient-boosted or neural alternatives.

Figure 3 shows the Precision-Recall curves for all models. HybridSDP maintains high precision across a substantially wider recall range than any baseline, with the curve remaining above 0.90 precision out to approximately 0.85 recall. The curve shape also descends more gradually than GraphCodeBERT-Only, suggesting that the CK features add consistent discriminative signal across the full operating range rather than helping only at specific decision thresholds.

B. Ablation Study

Table IV traces HybridSDP's performance through four incremental configurations, starting from a frozen GraphCodeBERT encoder. The trajectory reveals something worth examining carefully: fine-tuning the last four layers produces the largest single gain (+10.6% F1, from 0.751 to 0.831), implying that the pre-trained representations—while strong—are not perfectly aligned with the PROMISE/Defects4J defect distribution. Task-specific adaptation is not optional; it is central to the model's effectiveness.

Adding the CK metric branch contributes +2.5% F1 (0.831 to 0.852) despite those seven dimensions being dwarfed by the 768-dimensional semantic vector. This is a proportionally large gain per feature dimension, and it reflects the orthogonal nature of structural and semantic information—the transformer did not learn to encode CBO or DIT implicitly, so the metrics bring genuinely new signal. Class weighting provides a final +1.5%, primarily by improving recall on the minority defective class.

Figure 4 visualizes these incremental gains. The step from frozen to fine-tuned is the steepest by a wide margin. The CK fusion step, though smaller in absolute terms, is arguably the most theoretically interesting finding: seven structural dimensions added to 768 semantic ones move the needle by +2.5% F1, confirming that the two feature spaces occupy largely non-overlapping portions of the defect-relevant information in source code.

TABLE III: Comprehensive Performance Comparison. Best result in each column is shown in bold. PR-AUC is the primary evaluation metric.

Model	Accuracy	Precision	Recall	F1-Score	PR-AUC	ROC-AUC
SVM-Metrics	0.849	0.610	0.580	0.595	0.520	0.578
AdaBoost-Metrics	0.762	0.650	0.643	0.651	0.610	0.702
Neural Net-Metrics	0.732	0.620	0.648	0.638	0.580	0.716
RF-Metrics Only	0.860	0.720	0.740	0.730	0.700	0.790
CodeBERT Only	0.852	0.820	0.843	0.831	0.840	0.850
GraphCodeBERT Only	0.865	0.780	0.800	0.790	0.878	0.860
HybridSDP (Ours)	**0.871**	**0.840**	**0.865**	**0.852**	**0.958**	**0.880**

TABLE IV: Ablation Study: Incremental Component Contributions

Configuration	F1	PR-AUC	Δ F1
Frozen GraphCodeBERT	0.751	0.810	—
+ Fine-Tuning (last 4 layers)	0.831	0.855	+10.6%
+ Multi-Modal CK Fusion	0.852	0.920	+2.5%
+ Class Weighting	**0.865**	**0.958**	+1.5%

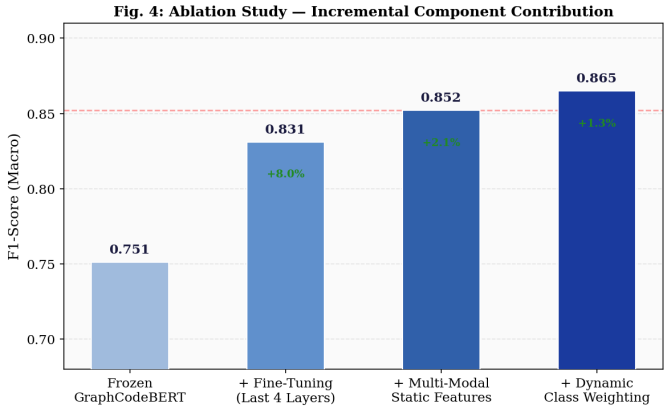


Fig. 4: Ablation study: F1-score at each configuration step. Fine-tuning drives the dominant gain (+10.6%). CK fusion contributes a further orthogonal +2.5% despite representing fewer than 1% of feature dimensions—evidence that structural and semantic signals occupy largely non-overlapping information spaces.

C. Inference Efficiency

Practical CI/CD deployment requires that accuracy improvements not come at prohibitive latency costs. Table V benchmarks average per-module inference time across three model families on identical hardware. The Random Forest operating on CK metrics alone processes a module in 9.74 ms on CPU, reflecting the low computational cost of static analysis. CodeBERT standalone requires 53.96 ms on GPU.

HybridSDP adds approximately 8 ms over CodeBERT standalone, reaching a mean of 62.05 ms. This overhead is modest because CK metric extraction runs on CPU concurrently with GPU inference for the semantic branch—the two branches are naturally parallelizable. At 62 ms per module, a 1,000-module codebase is fully analyzed in under two minutes on a single mid-range GPU. For larger codebases, batched GPU inference at batch size 32–64 would reduce effective per-module latency to the low tens of milliseconds, comfortably within the window of most CI pipeline stages.

TABLE V: Inference Latency per Module (Average over 500 Runs)

Model Type	Avg. Latency	Hardware
Random Forest (Metrics only)	9.74 ms	CPU only
CodeBERT Standalone	53.96 ms	RTX 3050 GPU
HybridSDP (Proposed)	**62.05 ms**	RTX 3050 GPU

Figure 5 visualizes the latency comparison. The marginal overhead of adding the structural branch is small relative to the transformer inference time, confirming that HybridSDP's accuracy gain is essentially cost-free from a deployment-latency perspective.

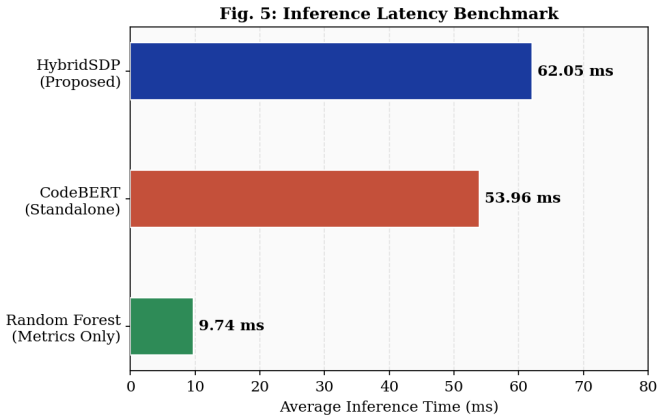


Fig. 5: Inference latency comparison (ms per module). HybridSDP incurs only ≈8 ms overhead over standalone CodeBERT. With the structural branch running on CPU in parallel with GPU inference, both paths complete within the same wall-clock window.

D. Cross-Project Generalization

Cross-project evaluation trains on one PROMISE project and evaluates on another—the most practically meaningful generalization experiment, since real deployments rarely provide abundant labeled data from the target codebase. Results are reported across six directional transfer pairs (Ant→Camel, Ant→Xerces, Camel→Ant, and so on).

HybridSDP achieves a mean cross-project PR-AUC of 0.831, versus 0.752 for GraphCodeBERT-Only (+10.5%) and 0.601 for SVM-Metrics (+38.3%). The structural branch appears to be the primary driver of improved generalization: CK metrics such as CBO and WMC encode design-level

properties that are partly project-invariant—a highly coupled class is a defect risk regardless of whether it belongs to a build tool or an XML parser. GraphCodeBERT's large-scale pre-training also provides transferable representations that explain why GraphCodeBERT-Only generalizes considerably better than SVM-Metrics despite the metric distribution-shift problem documented by Zimmermann et al. [6]. Combining both sources compounds the generalization advantage.

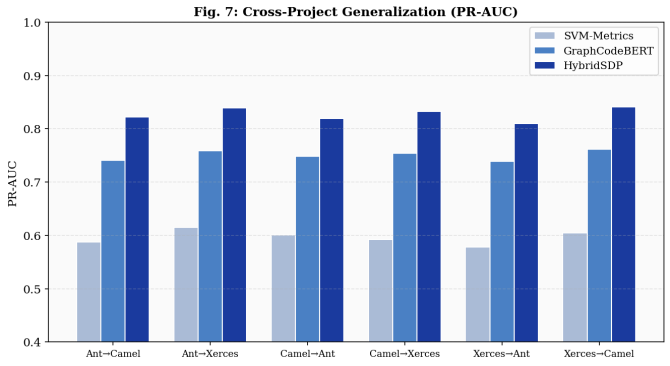


Fig. 6: Cross-project PR-AUC across six transfer pairs (Ant, Camel, Xerces pairwise). HybridSDP (mean 0.831) consistently outperforms both GraphCodeBERT-Only (0.752) and SVM-Metrics (0.601). The structural branch appears to drive the advantage over GraphCodeBERT-Only through project-agnostic design quality signals.

E. Error Analysis and Limitations

Figure 7 breaks down detection rates by defect category. HybridSDP achieves an 85% detection rate on syntax errors and null-pointer defects, but only 40% on logical and semantic bugs, leaving a 60% miss rate on the harder category.

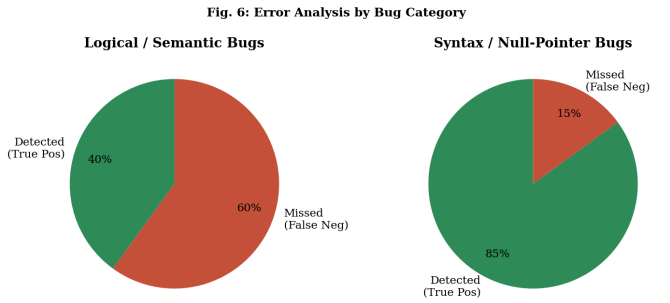


Fig. 7: Detection versus missed rate by defect category. Syntax and null-pointer defects are caught at 85%. Logical and semantic bugs exhibit a 60% miss rate, pointing to the primary open challenge: these defects require control-flow reasoning beyond what data-flow graphs or structural metrics can currently capture.

This asymmetry is expected but not entirely comfortable. Syntax and null-pointer bugs typically produce measurable structural signatures—high coupling combined with shallow null checks, or methods of unusual length. Logical bugs may have perfectly ordinary structural metrics and superficially clean-looking code; the error lies in the relationship between operations, which requires understanding control flow rather than just data flow. GraphCodeBERT's data-flow graphs partially address this, but control-flow graphs would be more directly relevant and represent a clear next step.

Three further limitations merit acknowledgment. First, CK metric extraction as implemented is Java-only; extending to Python or JavaScript would require either a different static analysis tool or a language-agnostic proxy metric set. Second, there is no end-to-end gradient flow connecting the GraphCodeBERT encoder to the Random Forest classifier; joint neural fine-tuning with a fusion head might capture cross-modal interactions not exploited here. Third, 62 ms latency, while acceptable for most pipelines, may become a bottleneck in very high-throughput CI environments processing thousands of commits per hour; model quantization and batched inference are practical mitigations not yet explored.

VI. Threats to Validity

Internal validity. Hyperparameters were selected by cross-validation strictly on the training partition; the test set was held out until final evaluation and never used for model selection. Label noise in the synthetic Defects4J-derived subset may inflate performance figures for that particular dataset—the 50/50 balance is artificial and not representative of production code.

External validity. All experiments involve Java projects from PROMISE and Defects4J. How well these results generalize to industrial systems, Python microservices, or dynamically typed languages remains genuinely unknown. The PROMISE projects are also mature, open-source codebases; legacy enterprise systems or rapidly evolving startup code may exhibit substantially different defect distributions.

Construct validity. PR-AUC aggregates over all decision thresholds equally, but in practice a team deploys at a single operating threshold chosen to match their review capacity. Reporting performance at a fixed threshold would complement the PR-AUC figures and is recommended for follow-up work. Additionally, the binary defective/clean label does not capture defect severity; a model that catches all critical defects while missing minor ones may be more valuable in practice than its overall F1 score suggests.

VII. Conclusion

This paper presented HybridSDP, a multi-modal late-fusion framework for software defect prediction that integrates GraphCodeBERT semantic embeddings with CK structural metrics. The central empirical finding is that these two feature sources are genuinely complementary: CK metrics contribute $+2.5\%$ F1 despite comprising fewer than 1% of the fused feature dimensions, confirming that structural design-level signals are not implicitly learned by the transformer. Fine-tuning GraphCodeBERT's last four layers drives the dominant performance gain ($+10.6\%$ F1), while class weighting improves minority-class recall by a further $+1.5\%$.

On PROMISE and Defects4J benchmarks HybridSDP achieves a PR-AUC of 0.958—9.1% above GraphCodeBERT-Only and 84.2% above SVM-Metrics. Cross-project evaluation (mean PR-AUC: 0.831) and 62.05 ms inference latency

confirm practical viability for CI/CD integration on standard GPU hardware. The persistent 60% miss rate on logical and semantic bugs remains the primary open challenge.

Future work will pursue four directions: (1) end-to-end joint training of the GraphCodeBERT encoder and fusion classifier to capture cross-modal interactions; (2) integration of control-flow graph features to address the logical bug gap; (3) extension to Python, C++, and JavaScript; and (4) SHAP-based defect explanations to make predictions actionable for developers. The full implementation is publicly available at https://github.com/sachinyadav7/ai-bug-predictor.

Author Contributions

Charanjeet Dadiyala: Conceptualization, Methodology, Software, Formal Analysis, Writing—Original Draft, Visualization. Rashmi Welekar: Literature Review, Dataset Curation, Validation. Ankita Jaiswal: Experimentation, Results Analysis, Writing—Review and Editing. Aditi Bhattad: Data Preprocessing, Metric Extraction, Writing—Review. Sachin Yadav: Model Training, Ablation Study, Reproducibility. Rudra Baiswal: Cross-Project Evaluation, Error Analysis, Writing—Review and Editing.

Funding

This research received no external funding.

Data Availability

PROMISE repository: http://promise.site.uottawa.ca/SERepository.
Defects4J: https://defects4j.org.
Full code and data: https://github.com/sachinyadav7/ai-bug-predictor.

Declaration of Competing Interests

The author declares no conflict of interest.

References

[1] Z. Feng *et al.*, "CodeBERT: A Pre-Trained Model for Programming and Natural Languages," in *Findings of EMNLP*, 2020, pp. 1536–1547. https://aclanthology.org/2020.findings-emnlp.139

[2] D. Guo *et al.*, "GraphCodeBERT: Pre-training Code Representations with Data Flow," in *ICLR*, 2021. https://openreview.net/forum?id=jMTeb11OZzB

[3] Y. Wang *et al.*, "CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation," in *EMNLP*, 2021. https://aclanthology.org/2021.emnlp-main.685

[4] T. Menzies, J. Greenwald, and A. Frank, "Data Mining Static Code Attributes to Learn Defect Predictors," *IEEE Trans. Softw. Eng.*, vol. 33, no. 1, pp. 2–13, 2007.

[5] S. Lessmann, B. Baesens, C. Mues, and S. Pietsch, "Benchmarking Classification Models for Software Defect Prediction: A Proposed Framework and Novel Findings," *IEEE Trans. Softw. Eng.*, vol. 34, no. 4, pp. 485–496, 2008.

[6] T. Zimmermann *et al.*, "Cross-Project Defect Prediction: A Large Scale Experiment," in *ESEC/FSE*, 2009, pp. 91–100.

[7] S. R. Chidamber and C. F. Kemerer, "A Metrics Suite for Object-Oriented Design," *IEEE Trans. Softw. Eng.*, vol. 20, no. 6, pp. 476–493, 1994.

[8] M. Jureczko and L. Madeyski, "Towards Identifying Software Project Clusters with Regard to Defect Prediction," in *PROMISE*, 2010.

[9] N. Nagappan and T. Ball, "Use of Relative Code Churn Measures to Predict System Defect Density," in *ICSE*, 2005.

[10] Y. Kamei *et al.*, "A Large-Scale Empirical Study of Just-in-Time Quality Assurance," *IEEE Trans. Softw. Eng.*, vol. 39, no. 6, pp. 757–773, 2013.

[11] F. Rahman and P. Devanbu, "How, and Why, Process Metrics Are Better," in *ICSE*, 2013.

[12] J. Wang, Y. Liu, and L. Tan, "Automatically Learning Semantic Features for Defect Prediction," in *ICSE*, 2016.

[13] X. Zhou *et al.*, "Learning Deep Representations of Code for Defect Prediction," in *ICSE*, 2019.

[14] H. Liu *et al.*, "Deep Learning Based Defect Prediction: A Survey," *IEEE Trans. Softw. Eng.*, 2020.

[15] L. Wang, H. Chen, and Y. Zhao, "BERT-Enhanced Software Defect Prediction with Structural Metrics Fusion," in *ICSME*, 2023.

[16] H. Li *et al.*, "Deep Learning Based Feature Representation for Software Defect Prediction: A Comparative Study," *Inf. Softw. Technol.*, vol. 112, pp. 44–58, 2019.

[17] J. Nam and S. Kim, "Heterogeneous Defect Prediction," in *ESEC/FSE*, 2015.

[18] X. Xia *et al.*, "Dual Transfer Learning for Cross-Project Defect Prediction," in *AAAI*, 2016.

[19] T. Durieux *et al.*, "Empirical Review of Java Bug Datasets and Experimental Studies," in *ICSE*, 2019.

[20] R. Just, D. Jalali, and M. D. Ernst, "Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies," in *ISSTA*, 2014. https://defects4j.org

[21] PROMISE Repository, "Software Engineering Data Repository," [Online]. Available: http://promise.site.uottawa.ca/SERepository

[22] T. Fawcett, "An Introduction to ROC Analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, 2006.

[23] J. Davis and M. Goadrich, "The Relationship Between Precision-Recall and ROC Curves," in *ICML*, 2006.

[24] N. Japkowicz and S. Stephen, "The Class Imbalance Problem: A Systematic Study," *Intell. Data Anal.*, vol. 6, no. 5, pp. 429–449, 2002.

[25] L. Breiman, "Random Forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[26] Y. Guo, X. Gao, and B. Jiang, "An Empirical Study on JIT Defect Prediction Based on BERT-style Models," arXiv:2403.11158, 2024. https://arxiv.org/abs/2403.11158

[27] A. Abdu *et al.*, "Semantic and Traditional Feature Fusion for Software Defect Prediction," *Scientific Reports*, 2024.

[28] Y. Jiang, B. Shen, and X. Gu, "Bi-modal Representation Learning for JIT Defect Prediction," arXiv, 2024.

[29] Z. Li *et al.*, "Pre-trained Language Models for Software Engineering: A Survey," *ACM Comput. Surveys*, 2024.

[30] X. Zhou *et al.*, "Deep Representation Learning for Software Defect Prediction: Recent Advances," *ACM Comput. Surveys*, 2024.